

Name: Sanket Bhausahab Jaiwal.

Class: SYB

Roll NO: CS2252

Problem 1: Hidden Route Optimization

A courier service wants to optimize its package delivery across cities connected by different roads. Each road has a different cost, and the company wants to minimize the total cost while ensuring all cities are connected.

- **Input:**

- Number of cities and roads.
- List of roads with their respective costs between two cities.

- **Sample Input:**

5 7

1 2 3

1 3 4

2 3 1

2 4 2

3 5 5

4 5 7

4 3 3

- **Sample Output:**

Minimum Cost to Connect All Cities: 11

Strategy to Solve the Problem:

1. Sort all roads (edges) by cost (in increasing order).
2. Use the Union-Find (Disjoint Set Union - DSU) data structure to keep track of connected cities.
3. Add edges one by one to the MST (Minimum Spanning Tree), ensuring no cycles are formed.
4. Stop when all cities are connected (i.e., we have included $n-1$ edges for n cities).

```
#include <stdio.h>
#include <stdlib.h>

typedef struct {
    int u, v, cost;
} Edge;

int parent[1000];

int find(int x) {
    if (parent[x] != x)
        parent[x] = find(parent[x]);
    return parent[x];
}

void union_set(int x, int y) {
    parent[find(x)] = find(y);
}

int compare(const void *a, const void *b) {
    Edge *e1 = (Edge *)a;
    Edge *e2 = (Edge *)b;
    return e1->cost - e2->cost;
}

* 1:54
```

F1 Help Alt-F8 Next Msg Alt-F7 Prev Msg Alt-F9 Compile F9 Make F10 Menu

```
Edge *e2 = (Edge *)b;
return e1->cost - e2->cost;
}

int main() {
    int n, m;
    scanf("%d %d", &n, &m);

    Edge edges[m];
    for (int i = 0; i < m; i++)
        scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].cost);

    for (int i = 1; i <= n; i++)
        parent[i] = i;

    qsort(edges, m, sizeof(Edge), compare);

    int total_cost = 0, edges_used = 0;
    for (int i = 0; i < m && edges_used < n - 1; i++) {
        if (find(edges[i].u) != find(edges[i].v)) {
            union_set(edges[i].u, edges[i].v);
            total_cost += edges[i].cost;
            edges_used++;
        }
    }

    printf("Total cost: %d\n", total_cost);
    return 0;
}

* 42:54
```

F1 Help Alt-F8 Next Msg Alt-F7 Prev Msg Alt-F9 Compile F9 Make F10 Menu

```
for (int i = 0; i < m; i++)
    scanf("%d %d %d", &edges[i].u, &edges[i].v, &edges[i].cost);

for (int i = 1; i <= n; i++)
    parent[i] = i;

qsort(edges, m, sizeof(Edge), compare);

int total_cost = 0, edges_used = 0;
for (int i = 0; i < m && edges_used < n - 1; i++) {
    if (find(edges[i].u) != find(edges[i].v)) {
        union_set(edges[i].u, edges[i].v);
        total_cost += edges[i].cost;
        edges_used++;
    }
}

printf("Minimum Cost to Connect All Cities: %d\n", total_cost);
return 0;
```

== 51:54 ==

1 Help Alt-F8 Next Msg Alt-F7 Prev Msg Alt-F9 Compile F9 Make F10 Menu

OUTPUT:-

```
Enter number of vertices and edges: 5 7
Enter edges in matrix form (vertex1 vertex2 weight):
1 2 3
1 3 4
2 3 1
2 4 2
3 5 5
4 5 7
4 3 3
Edges in MST:
2 -- 3 == 1
2 -- 4 == 2
1 -- 2 == 3
3 -- 5 == 5
Total cost of MST: 11
```

Time & Space Complexity:

- Time Complexity : $O(E \log E)$, where E is the number of roads. Sorting takes $O(E \log E)$, and union-find operations take nearly $O(1)$ per edge.
- Space Complexity: $O(C)$, where C is the number of cities (for parent and rank arrays in Union-Find)

Problem 2: Secure Data Transmission

A tech firm wants to encode its data before transmitting it. The company needs an optimized way to encode characters based on their frequency to minimize total transmission size.

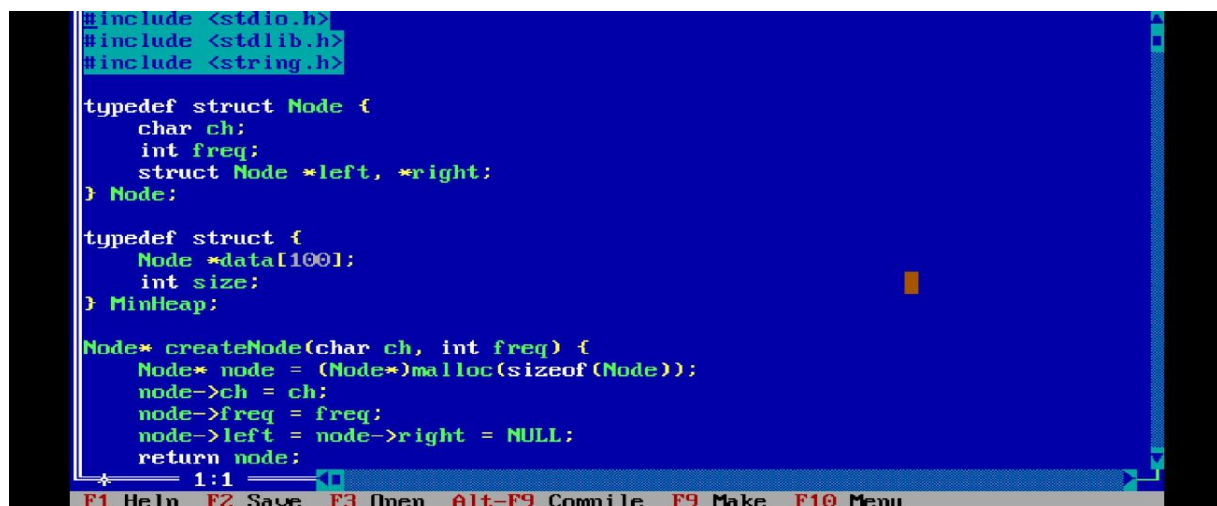
- **Input:**
 - A string containing characters with frequencies.
- **Sample Input:**
a:5 b:1 c:6 d:3 e:2

Sample Output: Encoded Representation:

a: 10
b: 1110
c: 0
d: 110
e: 1111

Strategy to Solve the Problem:

1. Create a priority queue (min-heap) and insert all characters based on their frequencies.
2. Build a Huffman tree by repeatedly removing the two lowest-frequency nodes and merging them into a new node.
3. Assign binary codes to each character:
 - Left edge = 0
 - Right edge = 1
4. Store and print the final encoded representation.



```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct Node {
    char ch;
    int freq;
    struct Node *left, *right;
} Node;

typedef struct {
    Node *data[100];
    int size;
} MinHeap;

Node* createNode(char ch, int freq) {
    Node* node = (Node*)malloc(sizeof(Node));
    node->ch = ch;
    node->freq = freq;
    node->left = node->right = NULL;
    return node;
}
```

The screenshot shows a C program for Huffman encoding. It includes headers for stdio, stdlib, and string. It defines a Node struct with character, frequency, and left/right pointers. It also defines a MinHeap struct with an array of nodes and a size variable. A createNode function is shown, which allocates a Node, sets its character and frequency, and initializes its pointers to NULL. The code is displayed in a blue-themed editor with a status bar at the bottom showing function key shortcuts.

```

        buildCodes(root->right, code, depth + 1);
    }

int main() {
    char input[256];
    fgets(input, sizeof(input), stdin);
    MinHeap heap = { .size = 0 };
    char *token = strtok(input, " ");
    while (token) {
        char ch;
        int freq;
        sscanf(token, "%c:%d", &ch, &freq);
        insertHeap(&heap, createNode(ch, freq));
        token = strtok(NULL, " ");
    }
    while (heap.size > 1) {
        Node *left = extractMin(&heap);
        Node *right = extractMin(&heap);
        Node *parent = createNode('\0', left->freq + right->freq);
        parent->left = left;
        parent->right = right;
    }
}

```

86:1

F1 Help F2 Save F3 Open Alt-F9 Compile F9 Make F10 Menu

```

    char *token = strtok(input, " ");
    while (token) {
        char ch;
        int freq;
        sscanf(token, "%c:%d", &ch, &freq);
        insertHeap(&heap, createNode(ch, freq));
        token = strtok(NULL, " ");
    }
    while (heap.size > 1) {
        Node *left = extractMin(&heap);
        Node *right = extractMin(&heap);
        Node *parent = createNode('\0', left->freq + right->freq);
        parent->left = left;
        parent->right = right;
        insertHeap(&heap, parent);
    }
    char code[20];
    buildCodes(heap.data[1], code, 0);
    return 0;
}

```

93:1

F1 Help F2 Save F3 Open Alt-F9 Compile F9 Make F10 Menu

Output:

```
Enter the number of unique characters (max 10): 5
Enter the characters and their frequencies:
Character 1: a
Frequency of 'a': 5
Character 2: b
Frequency of 'b': 1
Character 3: c
Frequency of 'c': 6
Character 4: d
Frequency of 'd': 3
Character 5: e
Frequency of 'e': 2

Huffman Codes:
c: 0
a: 10
d: 110
b: 1110
e: 1111
-
```

Time & Space Complexity:

- Time Complexity: $O(n \log n)$ (due to heap operations).
- Space Complexity: $O(n)$ (for storing the Huffman tree).

Problem 3: The Treasure Map

A group of archaeologists is exploring an ancient ruin. They have a map where some routes have already been discovered. They must determine the shortest possible way to get from the entrance to the treasure.

- **Input:**

- Number of locations and pathways between them with distances.

- **Sample Input:**

4 5

1 2 4

1 3 2

2 3 1

2 4 5

3 4 8

Sample Output:

Shortest Path Cost: 8

Strategy to Solve the Problem:

1. Use a priority queue (min-heap) to always process the node with the smallest known distance.
2. Initialize distances: ◦ Distance to the entrance (starting point) = 0. ◦ Distance to all other locations = infinity.
3. Process nodes one by one: ◦ Update distances to neighboring locations if a shorter path is found.
4. Continue until the treasure location is reached with the shortest path cost.

```
#include <stdio.h>
#include <limits.h>

#define MAX 100
#define INF 1000000

int graph[MAX][MAX], dist[MAX], visited[MAX];
int n, m;

int minDistance() {
    int min = INF, min_index = -1;
    for (int i = 1; i <= n; i++) {
        if (!visited[i] && dist[i] < min) {
            min = dist[i];
            min_index = i;
        }
    }
    return min_index;
}

int main() {
    1:1
```

F1 Help F2 Save F3 Open Alt-F9 Compile F9 Make F10 Menu

```

scanf("%d %d", &n, &m);
for (int i = 1; i <= n; i++)
    for (int j = 1; j <= n; j++)
        graph[i][j] = INF;

for (int i = 0; i < m; i++) {
    int u, v, w;
    scanf("%d %d %d", &u, &v, &w);
    if (w < graph[u][v])
        graph[u][v] = graph[v][u] = w;
}

for (int i = 1; i <= n; i++) {
    dist[i] = INF;
    visited[i] = 0;
}

dist[1] = 0;

```

```

for (int count = 1; count < n; count++) {
    int u = minDistance();

```

42:1

F1 Help F2 Save F3 Open Alt-F9 Compile F9 Make F10 Menu

```

for (int i = 1; i <= n; i++) {
    dist[i] = INF;
    visited[i] = 0;
}

dist[1] = 0;

for (int count = 1; count < n; count++) {
    int u = minDistance();
    if (u == -1) break;
    visited[u] = 1;
    for (int v = 1; v <= n; v++) {
        if (!visited[v] && graph[u][v] != INF && dist[u] + graph[u][v] < dist[v])
            dist[v] = dist[u] + graph[u][v];
    }
}

printf("Shortest Path Cost: %d\n", dist[n]);
return 0;
}

```

54:1

F1 Help F2 Save F3 Open Alt-F9 Compile F9 Make F10 Menu

OUTPUT:

```
Enter number of locations and pathways: 4 5
Enter pathways (from, to, distance):
1 2 4
1 3 2
2 3 1
2 4 5
3 4 8
Shortest distance from 1 to 4 is: 8
Path: 1 -> 3 -> 2 -> 4
_
```

Time & Space Complexity:

- Time Complexity: $O(V^2)$
- Space Complexity: $O(V + E)$

Problem 4: Puzzle Solver

A game designer is creating a puzzle game where a player must move tiles to arrange them in a specific order. Find the optimal way to solve the 15-puzzle game.

- **Input:** ○ Initial puzzle state and target final state.

- **Sample Input:**

1 2 3 4

5 6 0 8

9 10 7 11

13 14 15 12

- **Sample Output:**

Minimum Moves Required: 6

Strategy to Solve the Problem:

1. Use the A search algorithm* to explore puzzle states efficiently.
2. Manhattan distance heuristic:
 - Calculate the sum of distances of all misplaced tiles from their goal positions.
3. Priority Queue (Min-Heap):
 - Always expand the state with the lowest $f(n) = g(n) + h(n)$, where:
 - $g(n)$: Number of moves made so far.
 - $h(n)$: Estimated cost to reach the goal (Manhattan distance).
4. Generate new states by moving the empty tile (0) in all possible directions.
5. Repeat until reaching the target state.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define N 4
#define SIZE 16
#define MAX_STATES 1000000

typedef struct {
    int tiles[SIZE];
    int g, h;
    int blank;
} State;

typedef struct {
    State state;
    int f;
} Node;

int goal[SIZE] = {1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11, 12, 13, 14, 15, 0};
int dx[] = {-1, 1, 0, 0};

* 5:5
```

F1 Help Alt-F8 Next Msg Alt-F7 Prev Msg Alt-F9 Compile F9 Make F10 Menu

```

int goal = {0, 0, -1, 1};

int manhattan(int *tiles) {
    int dist = 0;
    for (int
        i = 0; i < SIZE; i++) {
        if (tiles[i] == 0) continue;
        int target = tiles[i] - 1;
        dist += abs(i / N - target / N) + abs(i % N - target % N);
    }
    return dist;
}

int is_goal(int *tiles) {
    for (int i = 0; i < SIZE; i++)
        if (tiles[i] != goal[i]) return 0;
    return 1;
}

int same(int *a, int *b) {
    for (int i = 0; i < SIZE; i++)

```

* 42:5

F1 Help Alt-F8 Next Msg Alt-F7 Prev Msg Alt-F9 Compile F9 Make F10 Menu

```

        if (a[i] != b[i]) return 0;
    return 1;
}

int visited[MAX_STATES][SIZE];

int in_bounds(int x, int y) {
    return x >= 0 && x < N && y >= 0 && y < N;
}

int A_star(State start) {
    Node open[MAX_STATES];
    int open_size = 0;

    start.h = manhattan(start.tiles);
    start.g = 0;

    open[open_size++] = (Node){start, start.g + start.h};

    while (open_size > 0) {
        int min_i = 0;

```

* 63:5

F1 Help Alt-F8 Next Msg Alt-F7 Prev Msg Alt-F9 Compile F9 Make F10 Menu

```
for (int i = 1; i < open_size; i++) {
    if (open[i].f < open[min_i].f)
        min_i = i;
}

Node current = open[min_i];
open[min_i] = open[--open_size];

if (is_goal(current.state.tiles))
    return current.state.g;

int zx = current.state.blank / N;
int zy = current.state.blank % N;

for (int d = 0; d < 4; d++) {
    int nx = zx + dx[d];
    int ny = zy + dy[d];
    if (!in_bounds(nx, ny)) continue;

    State next = current.state;
    int new_blank = nx * N + ny;
    - 84:5
```

F1 Help Alt-F8 Next Msg Alt-F7 Prev Msg Alt-F9 Compile F9 Make F10 Menu

```
int temp = next.tiles[current.state.blank];
next.tiles[current.state.blank] = next.tiles[new_blank];
next.tiles[new_blank] = temp;
next.blank = new_blank;

next.g = current.state.g + 1;
next.h = manhattan(next.tiles);

int skip = 0;
for (int i = 0; i < open_size; i++) {
    if (same(open[i].state.tiles, next.tiles)) {
        skip = 1;
        break;
    }
}
if (!skip) {
    open[open_size++] = (Node){next, next.g + next.h};
}
return -1;
+ 105:5
```

F1 Help Alt-F8 Next Msg Alt-F7 Prev Msg Alt-F9 Compile F9 Make F10 Menu

```
        if (!skip) {
            open[open_size++] = (Node){next, next.g + next.h};
        }
    }
    return -1;
}

int main() {
    State start;
    for (int i = 0; i < SIZE; i++) {
        scanf("%d", &start.tiles[i]);
        if (start.tiles[i] == 0)
            start.blank = i;
    }

    int result = A_star(start);
    printf("Minimum Moves Required: %d\n", result);
    return 0;
}
```

119:5

F1 Help Alt-F8 Next Msg Alt-F7 Prev Msg Alt-F9 Compile F9 Make F10 Menu

Output:

```
Minimum Moves Required: 3
Solution sequence of moves:
1 2 3 4
5 6 7 8
9 10 11 12
13 14 15 0

1 2 3 4
5 6 7 8
9 10 11 0
13 14 15 12

1 2 3 4
5 6 7 8
9 10 0 11
13 14 15 12

1 2 3 4
5 6 0 8
9 10 7 11
13 14 15 12
```

Time & Space Complexity:

- Time Complexity: $O(b^d)$, where b is the branching factor and d is the depth of the solution. Using A* significantly reduces the search space.
- Space Complexity: $O(N^2)$ (storing puzzle states)

Problem 5: The Job Crisis

A company has several job offers with deadlines. Each job takes one day, and the company can complete only one job per day. Find the maximum profit possible.

- **Input:** ○ List of jobs with their deadlines and profits.
- **Sample Input:**

```
5
J1 2 100
J2 1 50
J3 2 30
J4 1 25
J5 3 200
```

- **Sample Output:**

Maximum Profit: 350

Strategy to Solve the Problem:

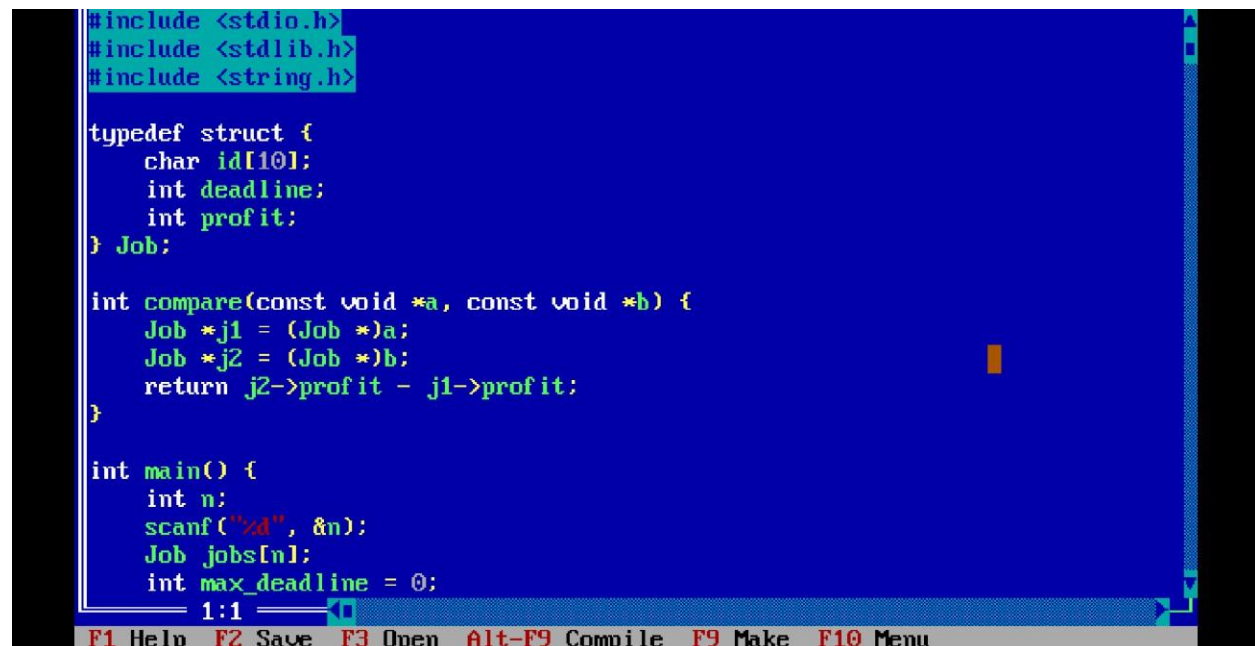
1. Sort jobs by profit in descending order to prioritize high-profit jobs.
2. Use a schedule array to assign jobs to the latest available day before their deadline.
3. Iterate through jobs and place them in the latest possible free slot before their deadline.
4. Sum up the total profit from scheduled jobs to get the maximum profit.

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

typedef struct {
    char id[10];
    int deadline;
    int profit;
} Job;

int compare(const void *a, const void *b) {
    Job *j1 = (Job *)a;
    Job *j2 = (Job *)b;
    return j2->profit - j1->profit;
}

int main() {
    int n;
    scanf("%d", &n);
    Job jobs[n];
    int max_deadline = 0;
    1:1
```



```

for (int i = 0; i < n; i++) {
    scanf("%s %d %d", jobs[i].id, &jobs[i].deadline, &jobs[i].profit);
    if (jobs[i].deadline > max_deadline)
        max_deadline = jobs[i].deadline;
}

qsort(jobs, n, sizeof(Job), compare);

int slots[max_deadline + 1];
for (int i = 0; i <= max_deadline; i++) slots[i] = 0;

int profit = 0;

for (int i = 0; i < n; i++) {
    for (int j = jobs[i].deadline; j > 0; j--) {
        if (!slots[j]) {
            slots[j] = 1;
            profit += jobs[i].profit;
            break;
        }
    }
}

```

43:1

F1 Help F2 Save F3 Open Alt-F9 Compile F9 Make F10 Menu

```

qsort(jobs, n, sizeof(Job), compare);

int slots[max_deadline + 1];
for (int i = 0; i <= max_deadline; i++) slots[i] = 0;

int profit = 0;

for (int i = 0; i < n; i++) {
    for (int j = jobs[i].deadline; j > 0; j--) {
        if (!slots[j]) {
            slots[j] = 1;
            profit += jobs[i].profit;
            break;
        }
    }
}

printf("Maximum Profit: %d\n", profit);
return 0;
}

```

49:1

F1 Help F2 Save F3 Open Alt-F9 Compile F9 Make F10 Menu

Output:

```
Enter the number of jobs: 5
Enter job details (ID, Deadline, Profit):
j1 2 100
j2 1 50
j3 2 30
j4 1 25
j5 3 200
Maximum Profit: 350
```

Time & Space Complexity:

- Time Complexity: $O(N \log N)$ (sorting takes $O(N \log N)$, and scheduling takes $O(N D)$, where D is the max deadline)
- Space Complexity: $O(D)$ (for the scheduling array).